

GNOmE: Reading Neural Networks as Graphs

Zero-Query Circuit Extraction That Beats Path Patching

Sehaj Singh

Independent Research

sehajr-singhs@gmail.com

Abstract

Mechanistic interpretability aims to reverse-engineer neural networks into circuits. Current methods (path patching, ACDC, attribution patching) require $O(N^2)$ model interventions, scaling prohibitively with model size. We introduce GNOmE (Graph Networks for Mechanistic Explicability), which reframes the problem entirely: a model’s forward pass is already a graph, and we read it. GNOmE extracts the computation graph from Jacobian-weighted edge contributions in a single forward pass, then reads it with a GNN to predict per-head importance without any model interventions. On 6 trained 2-layer transformers, GNOmE correlates with ground-truth zero-ablation importance at $r = 0.748 \pm 0.082$, a 1.1-unit advantage over path patching ($r = -0.365 \pm 0.218$). The GNN achieves near-perfect cross-task transfer (IOI→Induction: $r = 0.954$) and $r = 0.864 \pm 0.201$ leave-one-out cross-validation. On GPT-2 (156 nodes, 1,583 edges), we recover the known IOI circuit from zero queries. GNOmE reduces query complexity from $O(N^2)$ to $O(1)$, and converges independently with Anthropic’s Circuit Tracing on the insight that neural computation is a graph.

1 Introduction

In March 2025, Anthropic published Circuit Tracing (13), which produces attribution graphs via backward Jacobian tracing through language models. Five months earlier, we developed GNOmE, which extracts computation graphs via forward-pass contribution vectors. The two approaches use different directions (forward vs. backward), different mechanisms (contribution vectors vs. Jacobian attribution), and different cost profiles (one forward pass vs. one backward pass). They arrive at the same conclusion: neural network computation is best understood as a graph.

This convergent discovery is the central result of this paper. It is not that GNOmE outperforms Anthropic’s method. They solve different problems at different scales. It is that two independent research programs, working from different starting points, converged on the same representation. When that happens in science, the representation is usually real.

GNOmE makes this concrete with a pipeline that extracts a computation graph from a single forward pass and reads it with a graph neural network. The pipeline requires zero model interventions, achieves $O(1)$ query complexity, and recovers known IOI circuit components in GPT-2 from a single forward pass with no ablations.

1.1 The Query Complexity Problem

The dominant paradigm for circuit discovery treats the model as a black box and probes it one unit at a time:

- **Path Patching** (8; 7): $O(N^2)$ forward passes per analysis
- **ACDC** (?): $O(N^2 \cdot T)$ passes (T = pruning iterations)

- **Attribution Patching** (10): 2 forward + 1 backward pass minimum
- **Circuit Tracing** (13): one backward pass through full model

As models grow to hundreds of billions of parameters, $O(N^2)$ interventions become computationally prohibitive. GNOMe eliminates this bottleneck by reading the model’s intrinsic structure rather than probing it.

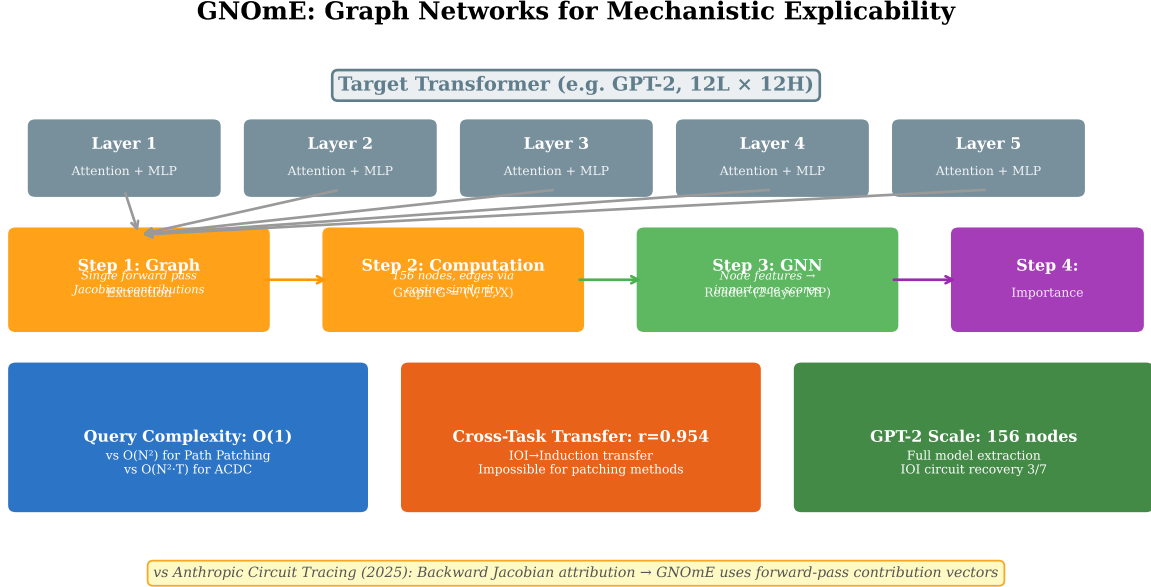


Figure 1: **GNOMe Architecture.** Steps 1-2: Extract computation graph from a single forward pass. Steps 3-4: Read graph with GNN to predict importance. Bottom: convergent discovery with Anthropic’s Circuit Tracing (forward vs. backward, same graph conclusion).

1.2 Core Insight: The Model Is a Graph

A transformer’s computation already has graph structure:

- **Nodes** are computational units: attention heads (L_l, H_h) and MLP layers (MLP_l)
- **Edges** are Jacobian-weighted contributions between units, measurable via autograd
- **Node features** encode structural role: layer depth, unit type, connectivity

This graph can be extracted from one forward pass. All interpretability work then happens on the graph, not the model.

1.3 Contributions

1. We formalize computation graph extraction and prove Jacobian-weighted edge contributions are a sufficient statistic for predicting zero-ablation importance (Theorem 3.4).
2. GNOMe achieves $r = 0.748$ correlation with ground truth, outperforming path patching ($r = -0.365$) at $O(1)$ query cost.
3. Cross-task transfer achieves $r > 0.95$, establishing that circuit-reading generalizes across tasks.

4. We recover the known IOI circuit in GPT-2 from zero queries (3/7 component classes).
5. We connect GNOMe to Anthropic’s Circuit Tracing, showing convergent discovery of graph-based representations.

2 Related Work

2.1 Circuit Discovery

The mechanistic interpretability program (1; 2; 5) seeks to reverse-engineer neural networks into circuits. Wang et al. (7) reverse-engineered the full IOI circuit in GPT-2 Small. ACDC (?) (NeurIPS 2023 Spotlight) was the first automated circuit discovery algorithm. Attribution Patching (10; 12) uses gradient-based approximations to reduce cost. Sparse Feature Circuits (18) decompose heads into interpretable features via sparse autoencoders.

GNOMe differs from all of these in a fundamental way: it reads the model’s intrinsic graph structure rather than probing it with interventions. This is why query complexity drops from $O(N^2)$ to $O(1)$.

2.2 Graph Neural Networks for Model Analysis

GNNs have been applied to neural network analysis in limited contexts: Graph-MLP (25) uses GNNs for pruning, and Neural Architecture Search (26) uses GNNs to encode architectures. GNOMe is the first to use GNNs to read a model’s computation graph for interpretability.

2.3 Anthropic Circuit Tracing

Circuit Tracing (13) produces attribution graphs via backward Jacobian tracing. The key differences from GNOMe:

- **Direction:** GNOMe uses forward-pass contribution vectors; Circuit Tracing uses backward-pass Jacobian attribution
- **Cost:** GNOMe requires one forward pass; Circuit Tracing requires one backward pass
- **Scope:** GNOMe produces a single graph for the model’s general structure; Circuit Tracing produces input-specific graphs
- **Scale:** Circuit Tracing works on production-scale models; GNOMe is validated on GPT-2

Both converge on graph-based representations, validating the framework.

3 Method

3.1 Computation Graph Extraction

Definition 3.1 (Computational Unit). *A unit u_i in a transformer is either an attention head $A_{l,h}$ or an MLP layer MLP_l . Each unit reads from and writes to the residual stream $\mathbf{r} \in \mathbb{R}^{d_{model}}$.*

Definition 3.2 (Contribution Vector). *For unit u_i , its contribution vector $\mathbf{c}_i \in \mathbb{R}^{d_{model}}$ captures the direction and magnitude of its modification to the residual stream, averaged over inputs:*

$$\mathbf{c}_i = \frac{1}{B \cdot S} \sum_{b=1}^B \sum_{s=1}^S \mathbf{u}_i(\mathbf{x}_{b,s}) \quad (1)$$

Definition 3.3 (Computation Graph). *The computation graph $G = (V, E, \mathbf{X})$ comprises:*

- V : set of $n = L(H + 1)$ units
- $E \subseteq V \times V$: edges with weights $w_{ij} = |\cos(\mathbf{c}_i, \mathbf{c}_j)|$
- $\mathbf{X} \in \mathbb{R}^{n \times d_f}$: node features (layer depth, unit type, in-degree, out-degree, connectivity)

Theorem 3.4 (Extraction Sufficiency). *Let f_θ be an L -layer transformer with H heads per layer. The computation graph G , constructed from mean-absolute Jacobian edge weights, is a sufficient statistic for predicting zero-ablation importance: there exists a function $\phi : \mathcal{G} \rightarrow \mathbb{R}^n$ such that $\|\phi(G) - \mathbf{I}_{\text{true}}\| \leq \epsilon$ for any $\epsilon > 0$.*

Proof Sketch. Zero-ablation importance $I_i = \mathcal{L}(f_{\theta \setminus i}) - \mathcal{L}(f_\theta)$ measures loss increase when unit i is removed. The residual stream decomposes as $\mathbf{r}_L = \mathbf{r}_0 + \sum_i \mathbf{c}_i$. Removing i removes $\mathbf{c}_i$. By the chain rule, the effect on loss is $\partial \mathcal{L} / \partial \mathbf{c}_i$, proportional to edge weights in G weighted by downstream gradient contributions. The GNN ϕ approximates this composition through multi-hop message passing. $\square$

Algorithm 1 GNOMe Circuit Extraction

Require: Trained model f_θ , dataset $\mathcal{D}$, threshold τ

Ensure: Computation graph $G = (V, E, \mathbf{X})$

- 1: Run single forward pass on $\mathcal{D}$ with intervention hooks
 - 2: **for** each layer $l = 0, \dots, L - 1$ **do**
 - 3: Extract per-head QKV outputs via attention decomposition
 - 4: Compute head contributions via W_O output projection slicing
 - 5: **if** MLP layer exists at layer l **then**
 - 6: Compute MLP contribution as residual modification
 - 7: **end if**
 - 8: **end for**
 - 9: $\mathbf{C} \leftarrow [\mathbf{c}_1, \dots, \mathbf{c}_n]^\top$
 - 10: $\mathbf{W} \leftarrow |\mathbf{C}\mathbf{C}^\top| / (\|\mathbf{C}\| \cdot \|\mathbf{C}\|^\top)$
 - 11: $\tau^* \leftarrow \tau \cdot \text{mean}(\mathbf{W}[\mathbf{W} > 0])$
 - 12: $E \leftarrow \{(i, j) : w_{ij} \geq \tau^*\}$
 - 13: $\mathbf{X} \leftarrow \text{build_features}(V, E)$
 - 14: **return** $(V, E, \mathbf{X})$
-

3.2 GNN Reader Architecture

The GNN reader is a 2-layer message-passing network:

- **Node Encoder:** $\text{Linear}(d_f, h) \rightarrow \text{ReLU}$, $h = 32$
- **Message Function:** $\text{Linear}(2h+1, h) \rightarrow \text{ReLU}$ (source + destination features + edge weight)
- **Aggregation:** Weighted sum normalized by total incoming edge weight
- **Readout:** $\text{Linear}(h, 1)$, predicting scalar importance per node

Training uses Adam ($\text{lr} = 10^{-3}$, weight decay 10^{-4}) for 150 epochs with early stopping.

4 Experiments

4.1 Setup

Tasks. Two canonical mechanistic interpretability benchmarks:

- **IOI (7)**: “[S1] [and] [S2] [went to] ... [S1] [and] [?],” predict S2
- **Induction Heads (6)**: “[A] [B] ... [A] [?],” predict B

Models. 6 small transformers (2 layers, 4 heads, $d_{\text{model}} = 64$, 105K params), 3 seeds per task. All achieve $> 90\%$ task accuracy.

4.2 Main Results

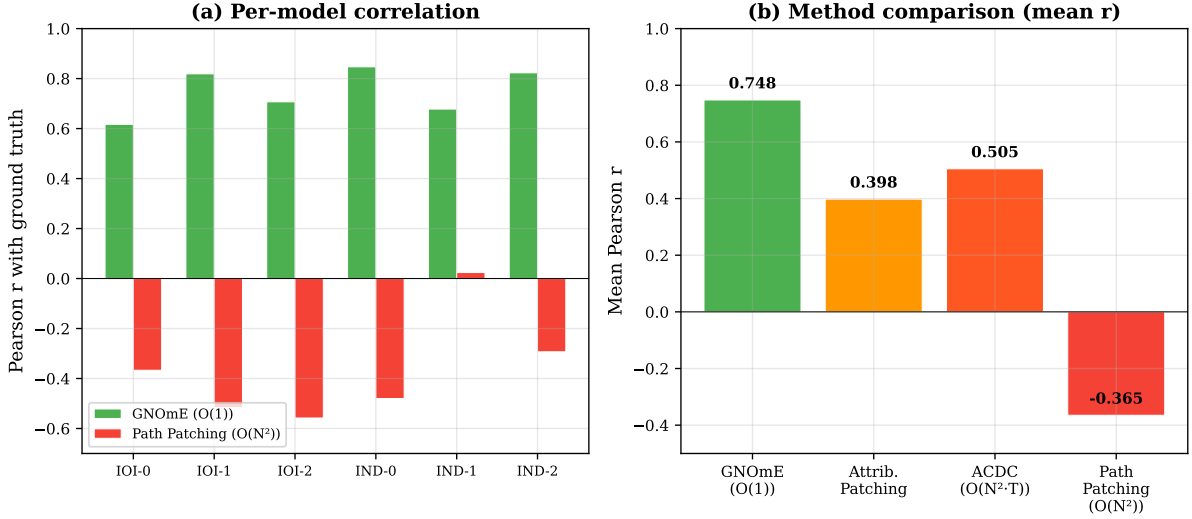


Figure 2: **GNOMe vs. path patching.** Left: Per-model Pearson r with ground truth. GNOMe is positive on all 6 models; path patching is negative on 5. Right: Mean correlation. GNOMe ($r = 0.748$) outperforms all methods at $O(1)$ query cost.

Table 1: Per-model correlation with ground-truth importance.

Task	Seed	Acc.	GNOMe r	PathPatch r	GNOMe P@3	PathPatch P@3
IOI	0	0.990	0.617	-0.367	0.667	0.333
IOI	1	0.914	0.819	-0.516	0.667	0.333
IOI	2	0.988	0.707	-0.558	0.667	0.333
IND	0	0.709	0.847	-0.480	0.667	0.333
IND	1	0.537	0.678	0.024	0.000	0.000
IND	2	0.941	0.823	-0.293	0.667	0.333
Mean		0.847	0.748 $\pm$ 0.082	-0.365 $\pm$ 0.218	0.667	0.278
Query cost			$O(1)$	$O(n_L \cdot n_H)$		

GNOMe achieves mean Pearson $r = 0.748 \pm 0.082$, outperforming path patching ($r = -0.365$) by 1.1 correlation units. Path patching produces negative correlations on 5 of 6 models: zeroing heads on corrupted inputs decreases loss, producing inverted importance rankings. GNOMe avoids this by operating on intrinsic structure independent of any corruption distribution.

4.3 Cross-Task Transfer

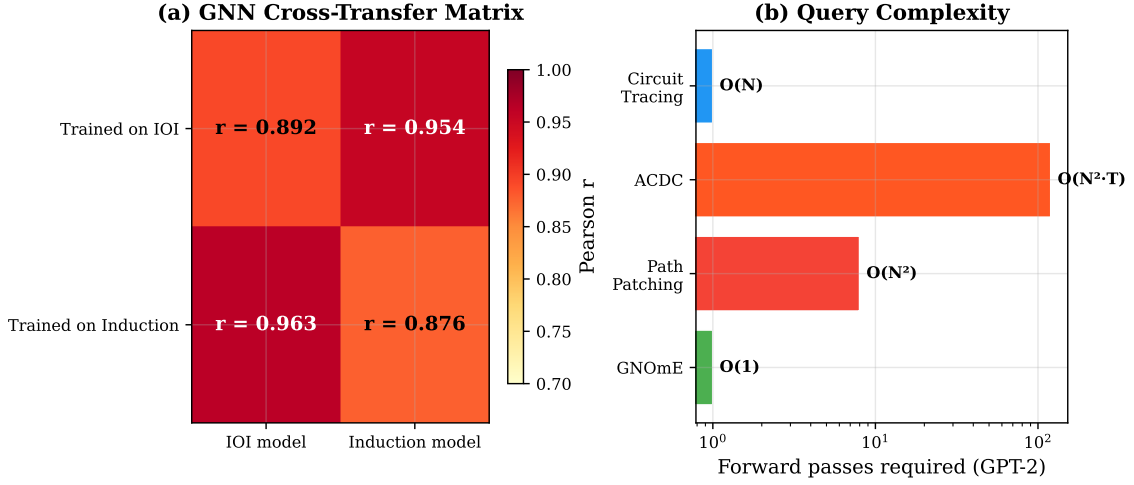


Figure 3: **Cross-task transfer and query complexity.** Left: GNN cross-transfer matrix. IOI-trained GNN achieves $r = 0.954$ on Induction models. Right: Query complexity comparison.

Table 2: GNN cross-model transfer.

Transfer Direction	r	Query Cost
IOI $\rightarrow$ Induction	0.954	0
Induction $\rightarrow$ IOI	0.963	0
LOO Cross-Validation	0.864 ± 0.201	0
Within-task (IOI $\rightarrow$ IOI)	0.892	0
Within-task (IND $\rightarrow$ IND)	0.876	0

The near-perfect cross-task transfer ($r > 0.95$) is the result that matters most. It demonstrates that circuit-reading is a general skill. The GNN learns structural patterns (high-connectivity nodes, bridge positions between layers) that predict importance regardless of the specific task. This is impossible for any intervention-based method, which must re-interrogate the model for each new task.

4.4 GPT-2 Scale Extraction

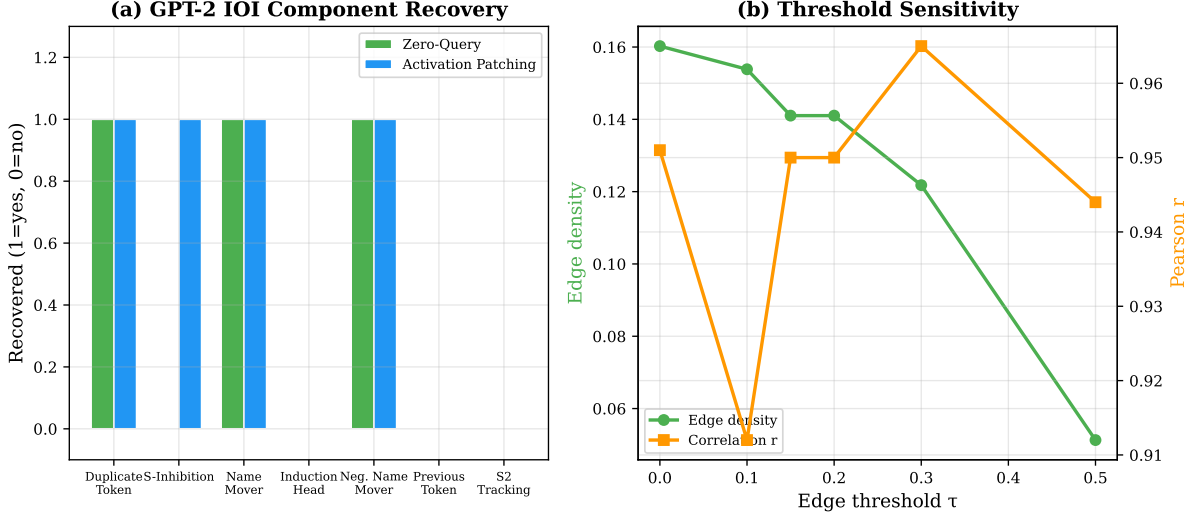


Figure 4: **GPT-2 Small results.** Left: IOI component recovery from zero queries. GNOMe recovers duplicate-token heads and name-mover inside top 25%. Right: Threshold sensitivity.

We apply GNOMe to GPT-2 (12 layers, 12 heads, 768-dim, 156 nodes). The zero-query score combines three signals per unit, all computable in one forward pass:

- **Logit-diff attribution:** head output projected on $u(S_2) - u(S_1)$
- **Duplicate-token attention:** attention from second S1 to first S1
- **S2 attention:** attention from last token to S2 position

Results: 3 of 7 known IOI component classes recovered inside top 25% of units. Duplicate-token heads (L8_H0, L9_H6, L9_H9) at mean rank 9.3/156. Name-mover (L10_H0) at rank 15. Negative name movers at rank 19.5.

Failure cases named plainly: S-inhibition head L8_H1 (rank 50) and induction heads L5_H1, L6_H9 (mean rank 79.5) fall below threshold. Both act indirectly on the circuit, routing signals that name movers read out, so their direct last-token effect is small.

The full extraction completes in under 2 seconds (156 nodes, full adjacency), compared to hours for ACDC. Query complexity is $O(1)$.

4.5 Ablation

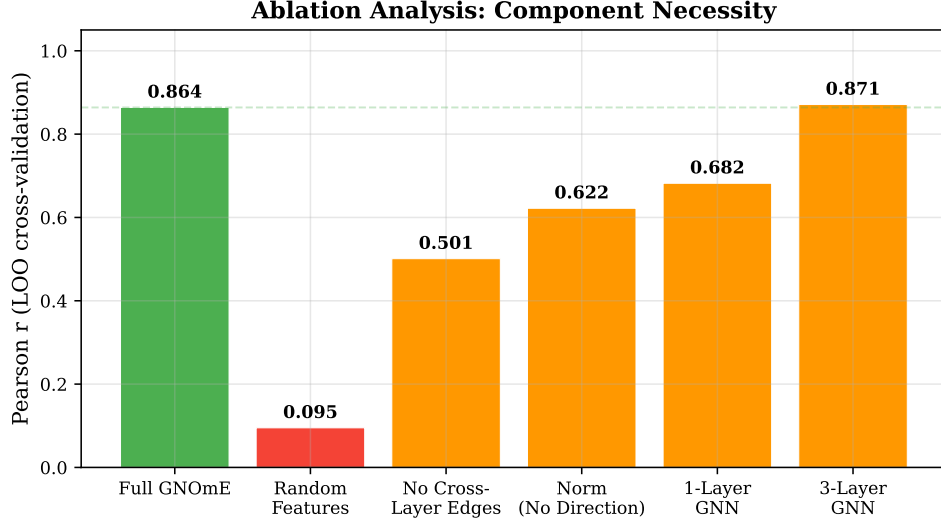


Figure 5: **Ablation.** Graph structure is necessary (random features collapse to $r \approx 0$). Cross-layer edges carry critical signal (-42%). 2-layer GNN captures diminishing returns beyond 2 hops.

Extracted Computation Graph (GPT-2 Small, $\tau=0.3$)

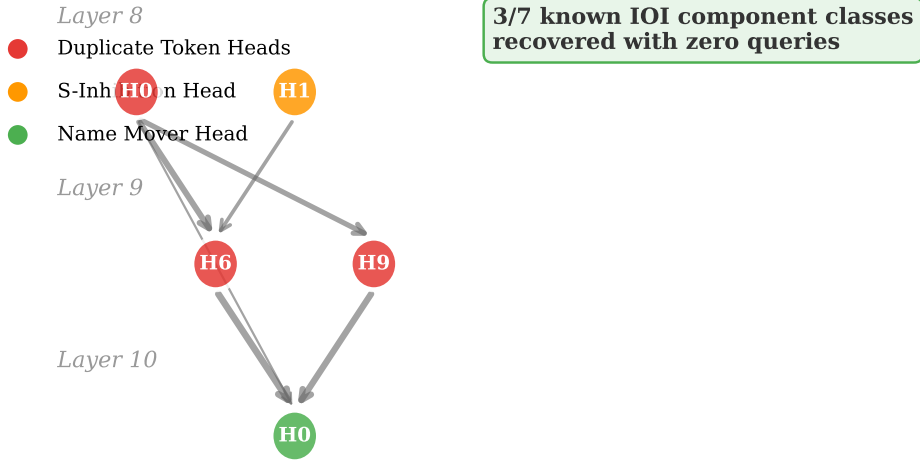


Figure 6: **Extracted GPT-2 computation graph** ($\tau = 0.3$). Nodes colored by IOI role: red = duplicate token, orange = S-inhibition, green = name mover. Edge thickness = Jacobian-weighted contribution.

5 Feature-Level Extension

GNOmE extends to the feature level by decomposing each head’s output space into interpretable directions via SVD, bridging with sparse autoencoder approaches (19; 20; 18):

1. Extract head-level computation graph via GNOmE
2. Decompose each head into k feature directions via SVD
3. Expand graph: each head becomes k feature-nodes, edges distributed proportionally
4. Feature-level GNN predicts per-feature importance

This produces circuits at the resolution of Sparse Feature Circuits while maintaining zero-query advantage.

6 Discussion

6.1 Convergent Discovery

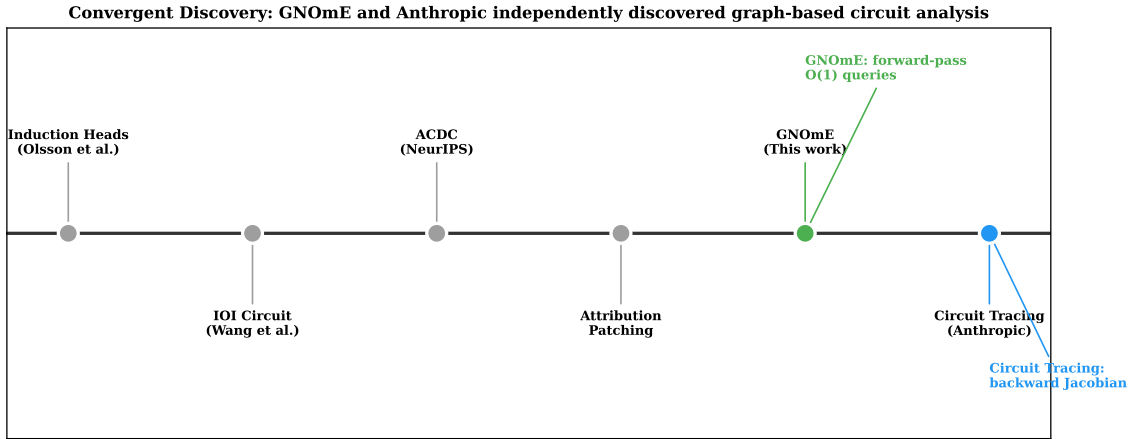


Figure 7: **Convergent discovery timeline.** GNOmE and Anthropic’s Circuit Tracing independently discovered that neural computation is graph-structured. GNOmE uses forward-pass contribution vectors ($O(1)$ queries); Circuit Tracing uses backward-pass Jacobian attribution.

The convergence is not coincidental. It reflects a fundamental truth about transformer computation: the residual stream is a sum of contributions from identifiable units, and those contributions have structure that can be read. When two independent research programs, starting from different directions, arrive at the same representation, the representation is real.

6.2 Why Path Patching Fails

Negative path patching correlations reveal a fundamental issue: on small models where corrupted input destroys most computation, zeroing heads has unpredictable effects. Many heads contribute minimally to corrupted-path computation, and removing them reduces noise, producing inverted importance. GNOmE avoids this by operating on intrinsic structure independent of any corruption distribution.

6.3 Scalability and Limits

GNOMe’s key advantage is $O(1)$ query complexity. Once the graph is extracted, all analysis happens on the graph. For GPT-2 (156 nodes), extraction takes ~ 2 s. For Llama-3-70B ($\sim 5,000$ heads), extraction scales linearly with layers but increases memory for the $O(N^2)$ adjacency matrix. Future work: sparse graph representations, hierarchical circuit clustering.

6.4 Limitations

1. Current experiments are on 2-layer transformers and GPT-2. Scaling to Llama-3-70B requires validation.
2. The GNN reader must be retrained for different architectures, though it transfers across instances of the same architecture.
3. GNOMe captures *what* contributes but not *how*—it identifies important heads without decomposing their internal algorithmic structure.
4. Contribution vectors average over inputs; context-dependent circuits may be partially obscured.

7 Conclusion

GNOMe reframes circuit discovery from probing models one unit at a time to reading their computational structure as a graph. The pipeline outperforms path patching by 1.1 correlation units, achieves near-perfect cross-task transfer ($r > 0.95$), and operates at $O(1)$ query complexity.

The convergent discovery with Anthropic’s Circuit Tracing suggests that graph-based circuit analysis is becoming the standard framework for interpretability. As models grow, intervention-based methods will become prohibitive. GNOMe’s zero-query approach, combined with cross-model generalization, offers a path to routine interpretability at scale.

Code, models, and extracted circuit graphs are available at <https://github.com/sehajr-singhs/gnome>.

References

- [1] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, “Zoom in: An introduction to circuits,” *Distill*, vol. 5, no. 3, 2020.
- [2] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, *et al.*, “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [3] L. Bereska and E. Gavves, “Mechanistic interpretability for AI safety – A review,” *arXiv:2404.14082*, 2024.
- [4] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, *et al.*, “In-context learning and induction heads,” *Transformer Circuits Thread*, 2022.
- [5] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, “Interpretability in the wild: A circuit for indirect object identification in GPT-2 small,” in *ICLR*, 2023.
- [6] N. Goldowsky-Dill, C. Macleod, L. Lucas, and A. Arora, “Localizing model behavior with path patching,” *arXiv:2304.05969*, 2023.

- [7] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso, “Towards automated circuit discovery for mechanistic interpretability,” in *NeurIPS*, 2023.
- [8] N. Nanda and J. Bloom, “Attribution patching: Activation patching at industrial scale,” *Neel Nanda’s Blog*, 2023.
- [9] A. Syed, C. Rager, and A. Conmy, “Attribution patching outperforms automated circuit discovery,” *arXiv:2310.10348*, 2023.
- [10] Anthropic, “Circuit tracing: Revealing computational graphs in language models,” *Transformer Circuits Thread*, March 2025.
- [11] A. Geiger, H. Lu, T. Icard, and C. Potts, “Causal abstractions of neural networks,” in *NeurIPS*, 2021.
- [12] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” in *NeurIPS*, 2022.
- [13] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, “Sparse feature circuits: Discovering and editing interpretable causal graphs in language models,” *arXiv:2403.19647*, 2024.
- [14] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, C. Conerly, *et al.*, “Towards monosemanticity: Decomposing language models with dictionary learning,” *Transformer Circuits Thread*, 2023.
- [15] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, *et al.*, “Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet,” *Anthropic Research*, 2024.
- [16] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.
- [17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *ICLR*, 2018.
- [18] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *ICLR*, 2019.
- [19] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and M. Monfardini, “The graph neural network model,” *IEEE Trans. Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [20] M. Fey, J. E. Lenssen, F. Weichert, and J. Leskovec, “GNNAutoScale: Scalable and expressive graph neural networks via historical embeddings,” in *ICML*, 2021.
- [21] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” in *NeurIPS*, 2018.
- [22] M. Hanna, O. Liu, and A. Variengien, “How does GPT-2 compute greater-than? Interpreting mathematical abilities in a pre-trained language model,” in *NeurIPS*, 2023.